

TRABAJO PRÁCTICO N°2

INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Desarrollo de Software

“CURL”

Docentes:

- Ing. Matias Cassani
- Ing. Matias Theiler

Alumnos:

Correo:

Legajo:

Ferreyra, Gaston	gf003432@gmail.com	16298
Martin, Jose Ignacio	joseignaciomartin@gmail.com	6895
Villoria, Federico Martin	fedevilloriaok@gmail.com	13906

- 1) A- Para el primer comando ponemos “curl {dirección url que queremos consultar}”, si hacemos curl <https://jsonplaceholder.typicode.com/posts> nos devuelve lo siguiente:

```
C:\Windows\System32>curl https://jsonplaceholder.typicode.com/posts
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas tota\nm\nnostrum rerum est autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis vo\nluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
  },
  {
    "userId": 1,
    "id": 3,
    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
    "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium qui\ns pariatur\nmolestiae porro eius odio et labore et velit aut"
  },
  {
    "userId": 1,
    "id": 4,
    "title": "eum et est occaecati",
    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic com\nmodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem rerum illo velit"
  }
]
```

Como se visualiza en la imagen estamos haciendo un GET a todos los objetos creados, encerrados entre las llaves {}, los cuales se ubican en un array de objetos, nos damos cuenta de esto ya que inicia la lista de objetos con corchetes []. Dentro de los objetos se puede observar las propiedades definidas con los atributos y sus valores. Lo cual nos devuelve código de estado HTTP 200 OK que significa que los datos de salida son los esperados.

B- Para consultar por aquellas publicaciones con id, agregamos la convención de pathparam para realizar una consulta de un post con cierto ID, en este caso utilizamos el 1, el cual nos devuelve únicamente el objeto de ID 1 y sus propiedades.

```
C:\Windows\System32>curl https://jsonplaceholder.typicode.com/posts/1
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
```

La estructura de la salida es esta:

```
{
  "userId": 1,
  "id": 1,
  "title": "...",
```

```
"body": "..."  
}
```

Donde:

userId: identifica al usuario dueño de la publicación.

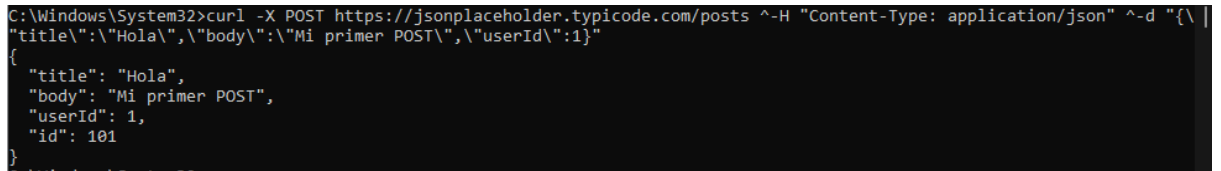
id: identificador único del post.

title: título.

body: contenido.

- 2) Para realizar un POST, en el caso de Windows hay que usar las barras invertidas “\” ya que sino no lo puede procesar. Dicho esto el comando es el siguiente:

```
curl -X POST https://jsonplaceholder.typicode.com/posts ^  
-H "Content-Type: application/json" ^  
-d "{\"title\": \"Hola\", \"body\": \"Mi primer POST\", \"userId\": 1}"
```



```
C:\Windows\System32>curl -X POST https://jsonplaceholder.typicode.com/posts ^-H "Content-Type: application/json" ^-d '{"title': "Hola",  
"body": "Mi primer POST",  
"userId": 1,  
"id": 101  
}'  
{  
  "title": "Hola",  
  "body": "Mi primer POST",  
  "userId": 1,  
  "id": 101  
}
```

Donde:

-X significa “Quiero crear/enviar datos”.

-H significa “Header” ahí le avisamos a la API que vamos a mandar en formato JSON.

-d significa “Body” o datos que enviamos, en este caso le enviamos un nuevo objeto con las propiedades mencionadas.

Si este comando fue exitoso nos devuelve un mensaje con el objeto creado. Puede depender del sistema operativo que se utilice, en nuestro caso que tenemos Windows, Linux y MacOS, las instrucciones son diferentes:

- **Para Linux:** `curl -X POST -H "Content-Type: application/json" -d '{
 "userId": 10,
 "title": "prueba",
 "body": "hola"
}'` <https://jsonplaceholder.typicode.com/posts>

- **Para MacOS:** `curl -X POST https://jsonplaceholder.typicode.com/posts \
-H "Content-Type: application/json" \
-d '{"title": "Mi titulo", "body": "Mi contenido", "userId": 1}'`

- 3) Para realizar un PUT ingresamos el siguiente comando: `curl -X PUT https://jsonplaceholder.typicode.com/posts/1 -H "Content-Type:`

```
application/json" -d '{"id":1,"title":"Titulo
modificado","body":"Contenido nuevo","userId":1}'
```

```
C:\Windows\System32>curl -X PUT https://jsonplaceholder.typicode.com/posts/1 -H "Content-Type: application/json" -d '{"
id":1,"title":"Titulo modificado","body":"Contenido nuevo","userId":1}'
{"id": 1,
"title": "Titulo modificado",
"body": "Contenido nuevo",
"userId": 1
}
```

En la URL indicamos con el id el posts que vamos a modificar, luego le cambiamos los valores de los atributos. Si el comando se introdujo correctamente, nos devolverá el objeto con los valores cambiados. La diferencia entre PUT y PATCH es que aquí (PUT) cambiamos el valor de TODOS los atributos.

- 4) Para realizar un PATCH. El comando es el siguiente:

```
curl -X PATCH https://jsonplaceholder.typicode.com/posts/1 \
-H "Content-Type: application/json" \
-d '{"title":"Nuevo titulo"}'
```

“aca en nuestro caso modificamos el campo titulo, pero podria ser cualquier otro”

```
gaston@MacBook-de-Gaston ~ % curl -X PATCH https://jsonplaceholder.typicode.com/posts/1 \
-H "Content-Type: application/json" \
-d '{"title":"Nuevo titulo"}'
{"userId": 1,
" id": 1,
" title": "Nuevo titulo",
" body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas
totam\nnostrum rerum est autem sunt rem eveniet architecto"
}
```

Básicamente utilizamos el patch para modificar parámetros específicos sin tener que requerir a modificar el recurso completo, sino el campo que nosotros especifiquemos.

- 5) Para realizar un DELETE. El comando es el siguiente:

```
curl -X DELETE https://jsonplaceholder.typicode.com/posts/101
```

```
}bellnet@bellnet-HP-Laptop-15-dy1xxx:~$curl -X DELETE https://jsonplaceholder.t
ypicode.com/posts/101
{}bellnet@bellnet-HP-Laptop-15-dy1xxx:~$
```

Donde:

-X especifica el método que luego ponemos que es DELETE.

Si este comando fue exitoso nos devuelve un mensaje con el objeto vacío {}.

REFLEXIÓN TÉCNICA

Responder de manera fundamentada:

1. ¿Por qué una API REST utiliza distintos métodos HTTP (**GET**, **POST**, **PUT**, **PATCH**, **DELETE**) en lugar de utilizar únicamente **POST**?

Principalmente por convención para mejorar la claridad de las interfaces de programación de aplicaciones (API) se utilizan distintos verbos y formas de enviar la información que buscan ser más explícitos a la hora de saber qué esperar al realizar una petición a un endpoint. A partir de eso es que también resulta en buenas prácticas para el manejo de la información ya que muchos servicios se encargan de dotar de seguridad según el método.

2. ¿Qué riesgos podrían existir si una API permitiera eliminar información utilizando el método **GET**?

El principal pensamos que puede ser que cause confusión y al querer desarrollar una interfaz que actúe para traer los datos de un objeto específico, los desarrolladores se basen en la convención y terminen eliminándolo por error. Además si se ejecuta desde un navegador es posible que quede en el historial y por error vuelva a ejecutarse. También al estar expuesto la identificación del objeto es permeable a ataques que eliminen otros objetos.